

KiCad

The KiCad Team

Table of Contents

Introduction	2
System Requirements	2
KiCad files and folders	2
Installing and Upgrading KiCad	5
Importing settings	5
Migrating files from previous versions	5
Using KiCad project manager	7
Project manager window	7
Project tree view	7
Side toolbar	8
Creating a new project	8
Importing a project from another EDA tool	9
KiCad configuration	10
Common preferences	10
Mouse and touchpad preferences	12
Hotkey preferences	13
Paths configuration	14
Libraries configuration	16
Project templates	17
Using templates	17
Template locations	18
Creating templates	18
Plugin and Content Manager	22
Actions reference	23
KiCad Project Manager	23

Reference manual

Copyright

This document is Copyright © 2010-2023 by its contributors as listed below. You may distribute it and/or modify it under the terms of either the GNU General Public License (<http://www.gnu.org/licenses/gpl.html>), version 3 or later, or the Creative Commons Attribution License (<http://creativecommons.org/licenses/by/3.0/>), version 3.0 or later.

All trademarks within this guide belong to their legitimate owners.

Contributors

Jean-Pierre Charras, Fabrizio Tappero, Jon Evans.

Feedback

The KiCad project welcomes feedback, bug reports, and suggestions related to the software or its documentation. For more information on how to submit feedback or report an issue, please see the instructions at <https://www.kicad.org/help/report-an-issue/>

Introduction

KiCad is an open-source software suite for creating electronic circuit schematics and printed circuit boards (PCBs). KiCad supports an integrated design workflow in which a schematic and corresponding PCB are designed together, as well as standalone workflows for special uses. KiCad also includes several utilities to help with circuit and PCB design, including a PCB calculator for determining electrical properties of circuit structures, a Gerber viewer for inspecting manufacturing files, and an integrated SPICE simulator for inspecting circuit behavior.

KiCad runs on all major operating systems and a wide range of computer hardware. It supports PCBs with up to 32 copper layers and is suitable for creating designs of all complexities. KiCad is developed by a volunteer team of software and electrical engineers around the world with a mission of creating free and open-source electronics design software suitable for professional designers.

The latest version of this documentation is available at <https://docs.kicad.org>.

System Requirements

KiCad is capable of running on a wide variety of hardware and operating systems, but some tasks may be slower or more difficult on lower-end hardware. For the best experience, a dedicated graphics card and display with 1920x1080 or higher resolution is recommended.

Please check the KiCad website for the latest system requirements: <https://kicad.org/help/system-requirements/>

KiCad files and folders

KiCad creates and uses files with the following specific file extensions (and folders) for schematic and board editing.

Project files

*.kicad_pro	Project file, containing settings that are shared between the schematic and PCB
*.pro	Legacy (KiCad 5.x and earlier) project file. Can be read and will be converted to a .kicad_pro file by the project manager.

Schematic editor files

*.kicad_sch	Schematic files containing all info and the components themselves.
*.kicad_sym	Schematic symbol library file, containing the component descriptions: graphic shape, pins, fields.
*.sch	Legacy (KiCad 5.x and earlier) schematic file. Can be read and will be converted to a .kicad_sch file on write.
*.lib	Legacy (KiCad 5.x and earlier) schematic library file. Can be read but not written.
*.dcm	Legacy (KiCad 5.x and earlier) schematic library documentation. Can be read but not written.
*-cache.lib	Legacy (KiCad 5.x and earlier) schematic component library cache file. Required for proper loading of a legacy schematic (.sch) file.
sym-lib-table	Symbol library table: list of symbol libraries available in the schematic editor.

Board editor files and folders

*.kicad_pcb	Board file containing all info but the page layout.
*.pretty	Footprint library folders. The folder itself is the library.
*.kicad_mod	Footprint files, containing one footprint description each.
*.kicad_dru	Design rules file, containing custom design rules for a certain .kicad_pcb file.
*.brd	Legacy (KiCad 4.x and earlier) board file. Can be read, but not written, by the current board editor.
*.mod	Legacy (KiCad 4.x and earlier) footprint library file. Can be read by the footprint or the board editor, but not written.
fp-lib-table	Footprint library table: list of footprint libraries available in the board editor.
fp-info-cache	Cache to speed up loading of footprint libraries. Does not need to be distributed with the project or put under version control.

Common files

*.kicad_prl	Local settings for the current project; helps Kicad remember the last used settings such as layer visibility or selection filter. May not need to be distributed with the project or put under version control.
*.kicad_wks	Page layout (drawing border and title block) description file
*.net	Netlist file created from the schematic, and read by the board editor. Note that the recommended workflow for transferring information from the schematic to the board does not require the use of netlist files.
*.cmp	Association between components used in the schematic and their footprints. It can be created by the Board Editor and imported by the Schematic Editor. Its purpose is to import changes from the board to the schematic, for users who change footprints in the Board Editor (for instance using Exchange Footprints command) and want to import these changes back to the schematic. recommended workflow for transferring information from the board to the schematic does not require the use of .cmp files.

Fabrication and documentation files

*.gbr	Gerber files, for fabrication.
*.drl	Drill files (Excellon format), for fabrication.
*.pos	Position files (ASCII format), for automatic insertion machines.
*.rpt	Report files (ASCII format), for documentation.
*.ps	Plot files (Postscript), for documentation.
*.pdf	Plot files (PDF format), for documentation.
*.svg	Plot files (SVG format), for documentation.
*.dxf	Plot files (DXF format), for documentation.
*.plt	Plot files (HPGL format), for documentation.

Storing and sending KiCad files

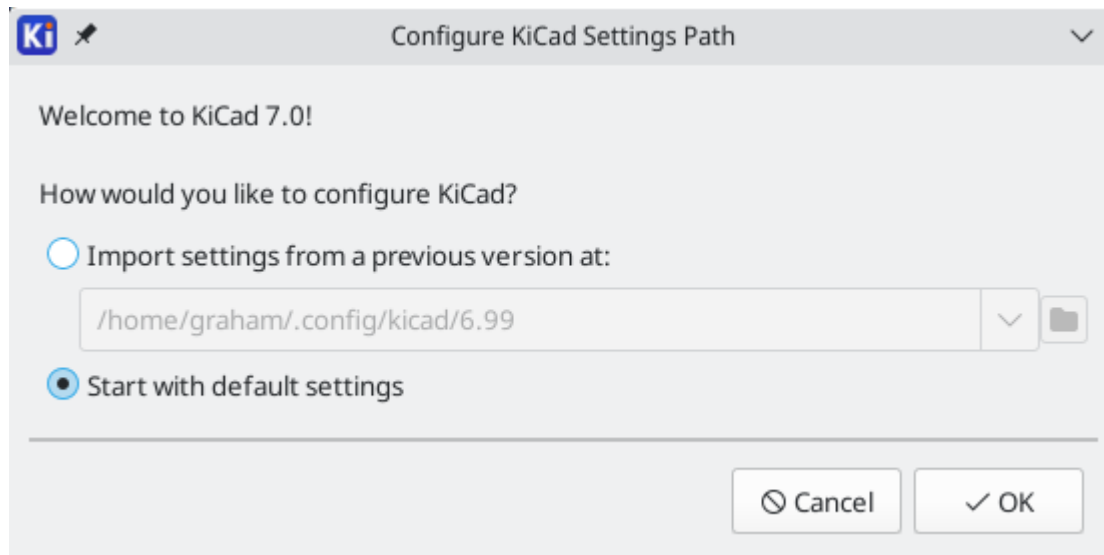
KiCad schematic and board files contain all the schematic symbols and footprints used in the design, so you can back up or send these files by themselves with no issue. Some important design information is stored in the project file (`.kicad_pro`), so if you are sending a complete design, make sure to include it.

Some files, such as the project-local settings file (`.kicad_prl`) and the `fp-info-cache` file, are not necessary to send with your project. If you use a version control system such as Git to keep track of your KiCad projects, you may want to add these files to the list of ignored files so that they are not tracked.

Installing and Upgrading KiCad

Importing settings

Each major release of KiCad has its own configuration, so that you may run multiple KiCad versions on the same computer without the configurations interfering. The first time you run a new version of KiCad, you will be asked how to initialize the settings:



If a previous version of KiCad is detected, you will have the option to import the settings from that version. The location of the previous configuration files is detected automatically, but you may override it to choose another location if desired.

Please note that, the schematic symbol and footprint library tables from the previous version of KiCad will **not** be imported.

You may also choose to start with default settings if you do not want to import settings from a previous version.

KiCad stores the settings files in a folder inside your user directory. Each KiCad version will store its settings in a subfolder of that folder (except for KiCad 5.1 and earlier, which did not use subfolders). Those folders are:

Windows	%APPDATA%\kicad
Linux	~/.config/kicad
macOS	/Users/<username>/Library/Preferences/kicad

Migrating files from previous versions

Modern versions of KiCad can open files created in earlier versions, but can only write files in the latest formats. This means that in general, there are no special steps to migrate files from a previous version besides opening the files. In some cases, the file extension for a file has changed from one KiCad version to the next. After opening these files, they will be saved in the new format with the new file extension. The old files will not be deleted automatically.

In general, files created or modified by one version of KiCad **cannot** be opened by older versions of KiCad. For this reason, it is important to keep backup copies of your projects when testing a new KiCad release, until you are confident that you will not need to use the older KiCad version anymore.

NOTE

Hotkey configurations are not imported from previous versions at this time. You can manually import hotkey configurations by copying the various `*.hotkeys` files from the old version configuration directory to the new one. If you do so, please note that KiCad will not automatically detect conflicts such as one key being assigned to multiple actions.

Using KiCad project manager

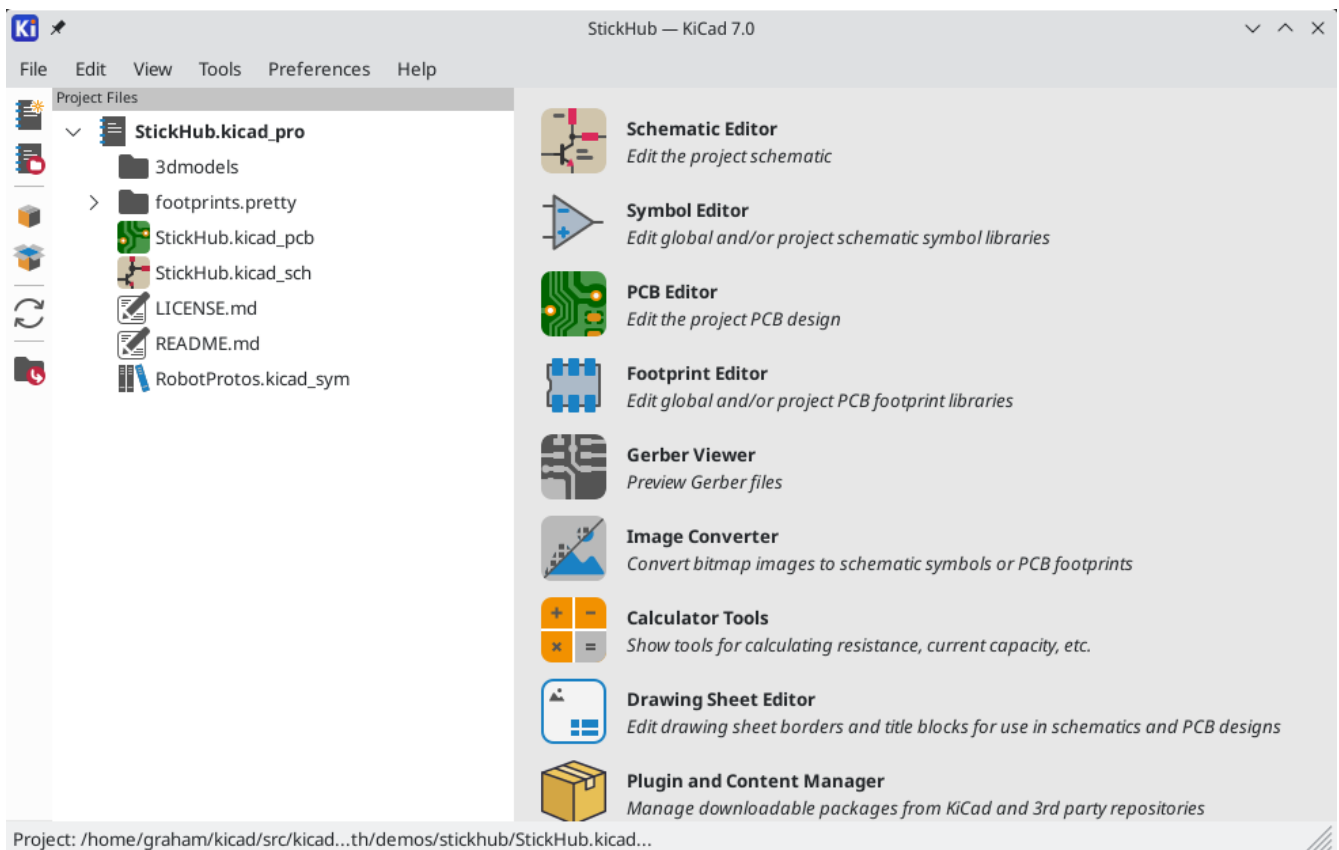
KiCad project manager (kicad or kicad.exe) is a tool which can easily run the other tools (schematic and board editors, Gerber viewer and utility tools) when creating a design.

Running the other tools from KiCad manager has some advantages:

- cross probing between schematic editor and board editor.
- synchronization of the design between the schematic editor and board editor (without creating netlist files)

KiCad currently only supports having one project open at a time. When running the schematic and board editors from the KiCad project manager, you can only edit the schematics and board associated with the open project. When these tools are run in *standalone* mode, you can open any file in any project, but cross probing between tools can give strange results.

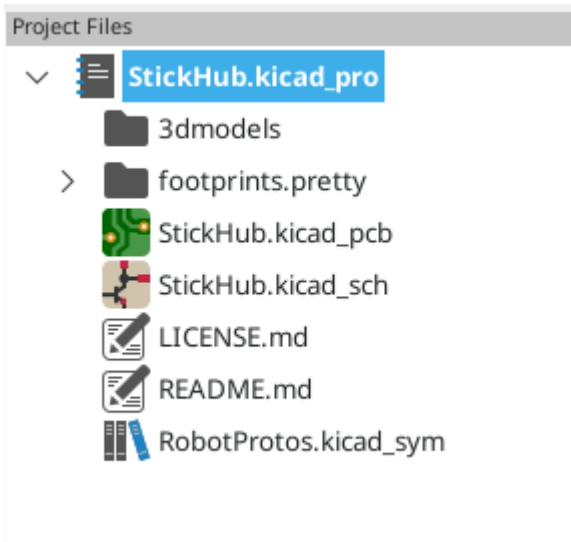
Project manager window



The KiCad project manager window is composed of a tree view on the left showing the files associated with the open project, and a launcher on the right containing shortcuts to the various editors and tools.

Project tree view

The tree view shows a list of files inside the project folder. Double-clicking on a file in the tree view will open it in the associated editor. Right-clicking on a file will open a context menu with some file manipulation commands.



NOTE Only files that KiCad understands how to open are displayed in the project tree view.

Side toolbar

The toolbar on the left side of the window provides shortcuts for common project operations:

	Create a new project.
	Open an existing project.
	Create a zip archive of the whole project. This includes schematic files, libraries, PCB, etc.
	Extract a project zip archive into a directory. Files in the destination directory will be overwritten.
	Refresh the tree view, to detect changes made on the filesystem.
	Open the project working directory in a file explorer.

Creating a new project

Most KiCad designs start with the creation of a project. There are two ways to create a project from the KiCad project manager: you may create an empty project, or create a project based on an existing template. This section will cover the creation of a new, empty project. Creating projects from templates is covered in the [Project Templates](#) section.

To create a new project, use the **New Project...** command in the **File** menu, click the **New Project** button in the top toolbar, or use the keyboard shortcut (**Ctrl+N** by default).

You will be prompted for a name to give your project. By default, a directory will be created for your project with the same name. For example, if you enter the name `MyProject`, KiCad will create the directory `MyProject` and the project file `MyProject/MyProject.kicad_pro` inside it.

If you already have a directory to store your project files in, you can uncheck the *Create a new directory for the project* checkbox in the **New Project** dialog.

NOTE

It is strongly recommended that you store each KiCad project inside its own directory.

Once you select the name of your project, KiCad will create the following files inside the project directory:

example.kicad_pro	KiCad project file.
example.kicad_sch	Main schematic file.
example.kicad_pcb	Printed circuit board file.

Importing a project from another EDA tool

KiCad is able to import files created by some other software packages. Currently the following types of project are supported:

*.sch, *.brd	Eagle 6.x or newer (XML format)
*.csa, *.cpa	CADSTAR archive format

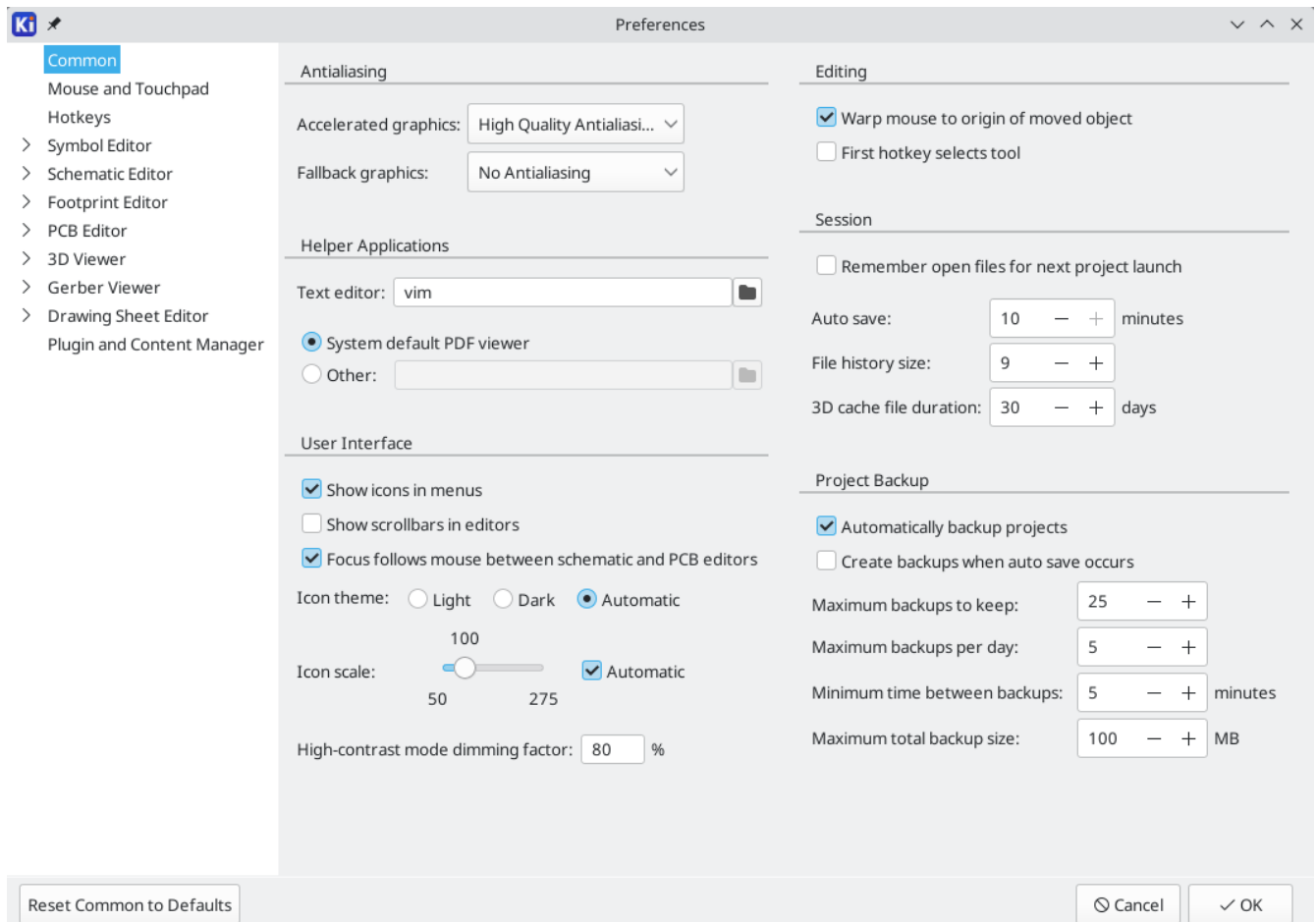
To import a project from one of these tools, choose the appropriate option from the **Import Non-KiCad Project** submenu of the **File** menu.

You will be prompted to select either a schematic or a board file in the import file browser dialog. The imported schematic and board files should have the same base file name (e.g. project.sch and project.brd). Once the requested files are selected, you will be asked to select a directory to store the resulting KiCad project.

KiCad configuration

The KiCad preferences can always be accessed from the **Preferences** menu, or by using the hotkey (default `Ctrl+,`). The Preferences dialog is shared between the running KiCad tools. Some preferences apply to all tools, and some are specific to a certain tool (such as the schematic or board editor).

Common preferences



Accelerated graphics antialiasing: KiCad can use different methods to prevent aliasing (jagged lines) when rendering using a graphics card. Different methods may look better on different hardware, so you may want to experiment to find the one that looks best to you.

Fallback graphics antialiasing: KiCad can also apply antialiasing when using the fallback graphics mode. Enabling this feature may result in poor performance on some hardware.

Text editor: Choose a text editor to use when opening text files from the project tree view.

PDF viewer: Choose a program to use when opening PDF files.

Show icons in menus: Enables icons in drop-down menus throughout the KiCad user interface.

NOTE | Icons in menus are not displayed on some operating systems.

Show scrollbars in editors: When enabled, scrollbars are displayed next to the editing canvases in each tool. When disabled, scrollbars are not shown.

Focus follows mouse between schematic and PCB editors: When enabled, the window under the mouse cursor will automatically become focused.

Icon scale: Sets the size of the icons used in menus and buttons throughout KiCad. Choose *Automatic* to pick an appropriate icon scale automatically based on your operating system settings.

Icon theme: Sets whether to use the icon theme designed for light window backgrounds or dark window backgrounds. The default setting of Automatic will choose the theme based on the lightness of the operating system window theme.

High-contrast mode dimming factor: Sets how much non-focused items are dimmed in high-contrast display mode.

Warp mouse to origin of moved object: When enabled, the mouse cursor will be repositioned (warped) to the origin of an object when you start a move command on that object.

First hotkey selects tool: When disabled, pressing the hotkey for a command such as *Add Wire* will immediately start the command at the current cursor location. When enabled, pressing the hotkey the first time will just select the *Add Wire* tool but will not immediately begin a wire.

Remember open files for next project launch: When enabled, KiCad will automatically re-open any files that were previously open when a project is re-opened.

Auto save: When editing schematics and board files, KiCad can automatically save your work periodically. Set to 0 to disable this feature.

File history size: Configure the number of entries in the list of recently-opened files

3D cache file duration: KiCad creates a cache of 3D models in order to speed up the 3D viewer. You can configure how long to keep this cache before deleting old files.

Automatically backup projects: When enabled, KiCad projects will be archived to ZIP files automatically according to the settings below. The archives will be stored in a subfolder of the project folder. Backups are created when saving files in the project.

Create backups when auto save occurs: When enabled, a backup will be created every time an automatic file save occurs (if the backup is permitted by the settings below). This setting has no effect if the auto save interval is set to 0 (disabled).

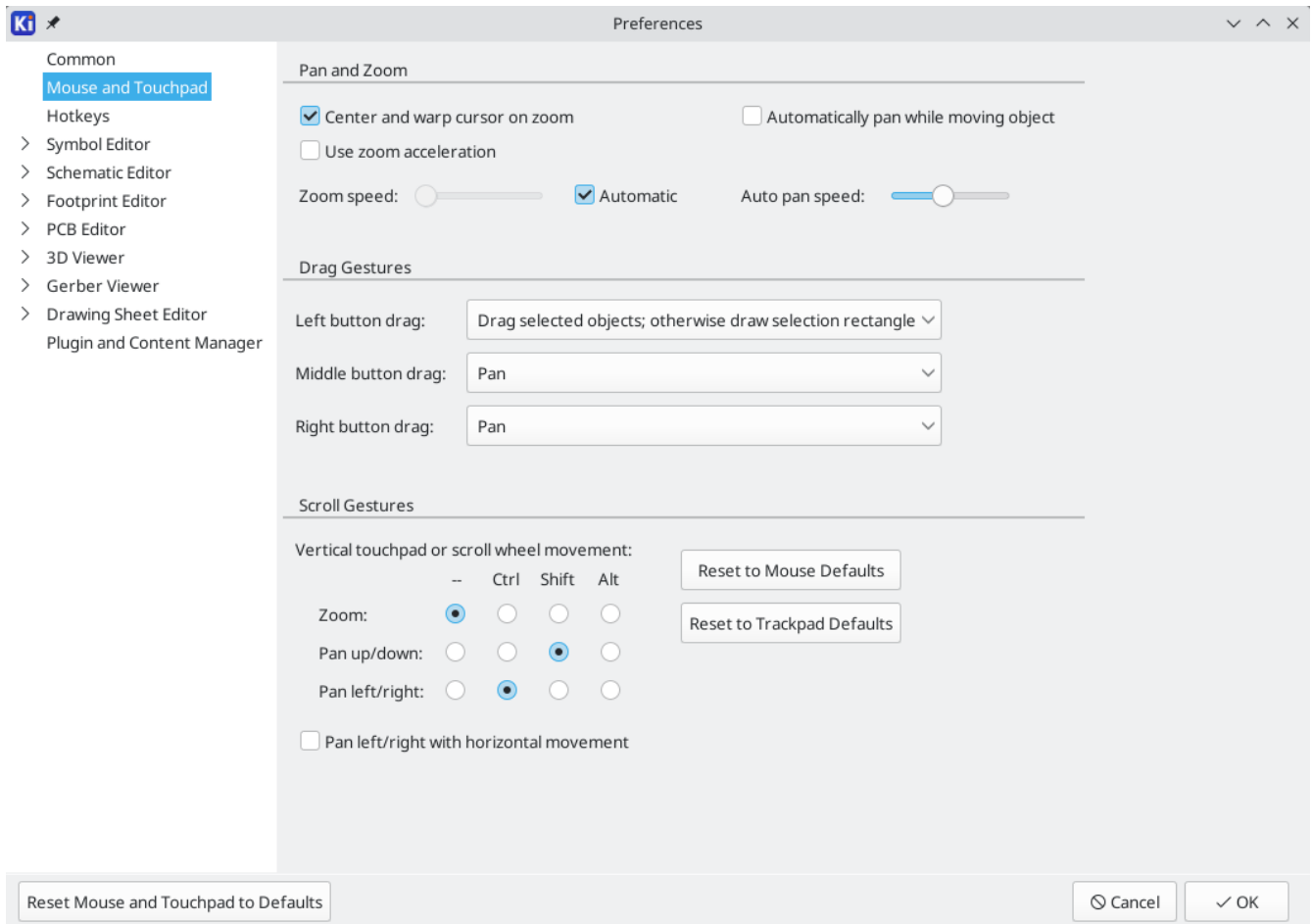
Maximum backups to keep: When creating a new backup, the oldest backup file will be deleted to keep the total number of backup files below this limit.

Maximum backups per day: When creating a new backup, the oldest backup file created on the current day will be deleted to stay below this limit.

Minimum time between backups: If backup is triggered (for example, by saving a board file), the backup will not be created if an existing backup file is newer than this limit.

Maximum total backup size: When creating a new backup file, the oldest backup files will be deleted to keep the total size of the backup files directory below this limit.

Mouse and touchpad preferences



Center and warp cursor on zoom: When enabled, zooming using the hotkeys or mouse wheel will cause the view to be centered on the cursor location.

Use zoom acceleration: When enabled, scrolling the mouse wheel or touchpad faster will cause the zoom to change faster.

Zoom speed: Controls how much the zoom changes for a given amount of scrolling the mouse wheel or touchpad. Use *Automatic* to set a default value depending on your operating system.

Automatically pan while moving object: When enabled, the view can be panned while moving an object by moving close to the edge of the canvas.

Auto pan speed: Controls how fast the canvas pans while moving an object.

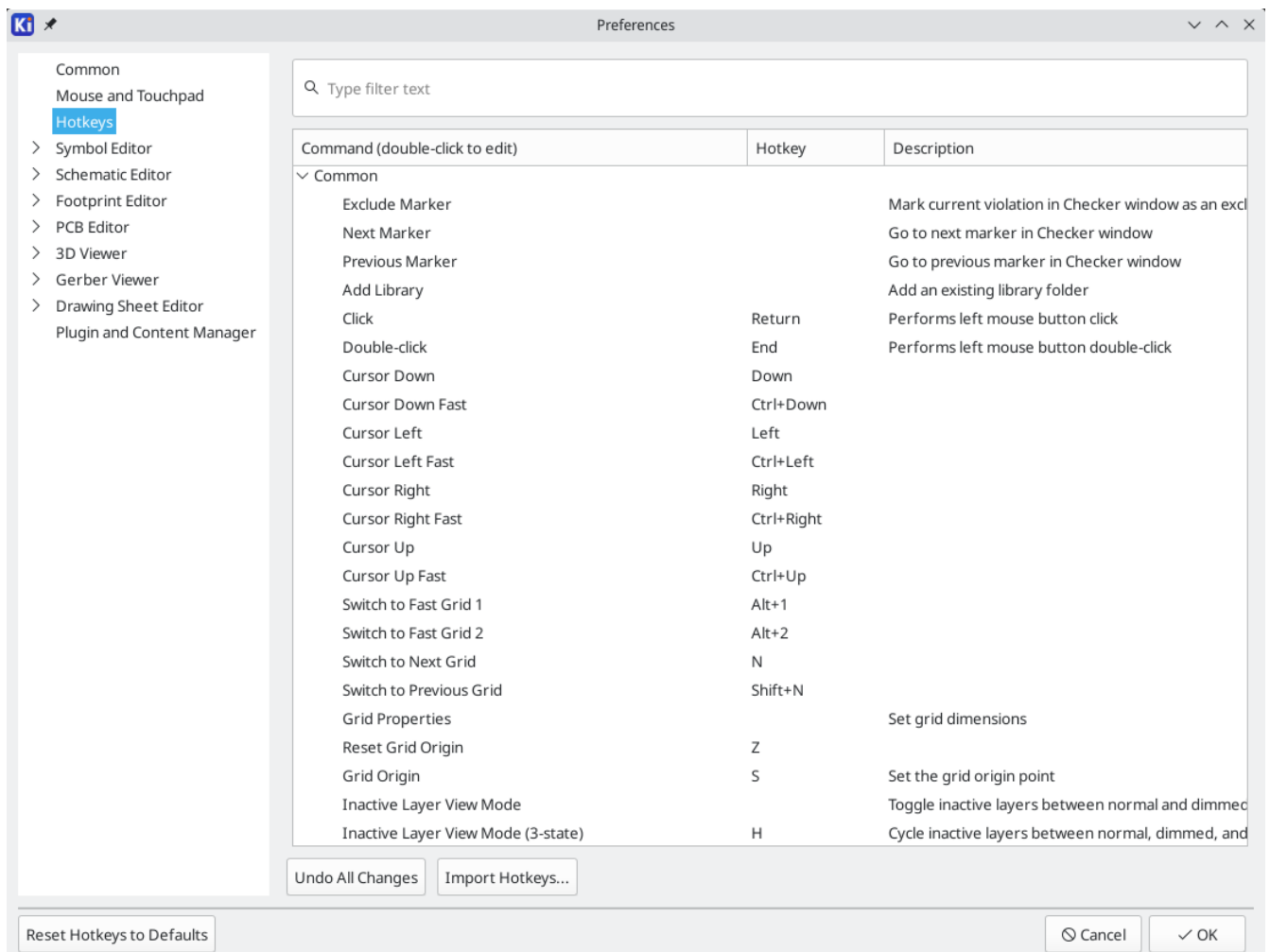
Mouse buttons: You can set the behavior of dragging the middle and right mouse buttons to zoom the view, pan the view, or have no effect. You can also set the behavior of dragging the left mouse button depending on whether or not any objects are already selected in the editing canvas.

NOTE | The left mouse button is always used for selecting and manipulating objects.

Mouse wheel and touchpad scrolling: You can set the behavior of scrolling the mouse wheel or vertical motion of the touchpad while pressing certain modifier keys.

Pan left/right with horizontal movement: When enabled, you can pan the view using the touchpad or horizontal scroll wheel (if present on your mouse).

Hotkey preferences



You can use this dialog to customize the hotkeys used to control KiCad. The hotkeys in the *Common* section are shared between every KiCad program. Hotkeys for each specific KiCad program are shown when that program is running. You can assign the same hotkey to a different action in different KiCad programs (for example, the schematic editor and the board editor), but you cannot assign a hotkey to more than one action in the same program.

There are many available commands, and so not all of them have a hotkey assigned by default. You can add a hotkey to any command by double-clicking on the command in the list. If you choose a hotkey that is already assigned to a different command, you can choose to use that hotkey on your chosen command, which will remove the hotkey assignment from the conflicting command.

Changes that you have made to hotkey assignments are shown with a ***** character at the end of the command name. You can undo changes to a specific command by right-clicking that command and selecting **Undo Changes**, or you can undo all changes with the button below the command list.

Importing hotkeys

Hotkey preferences are stored in `.hotkeys` files in the KiCad settings directory (see the [Settings](#) section for information about where the settings directory is on your operating system). If you have configured KiCad

hotkeys the way you like on one computer, you can transfer that configuration to another computer by importing the appropriate `.hotkeys` file(s).

Paths configuration

In KiCad, one can define paths using an **environment variable**. A few environment variables are internally defined by KiCad, and can be used to define paths for libraries, 3D shapes, etc.

This is useful when absolute paths are not known or are subject to change (e.g. when you transfer a project to a different computer), and also when one base path is shared by many similar items. Consider the following which may be installed in varying locations:

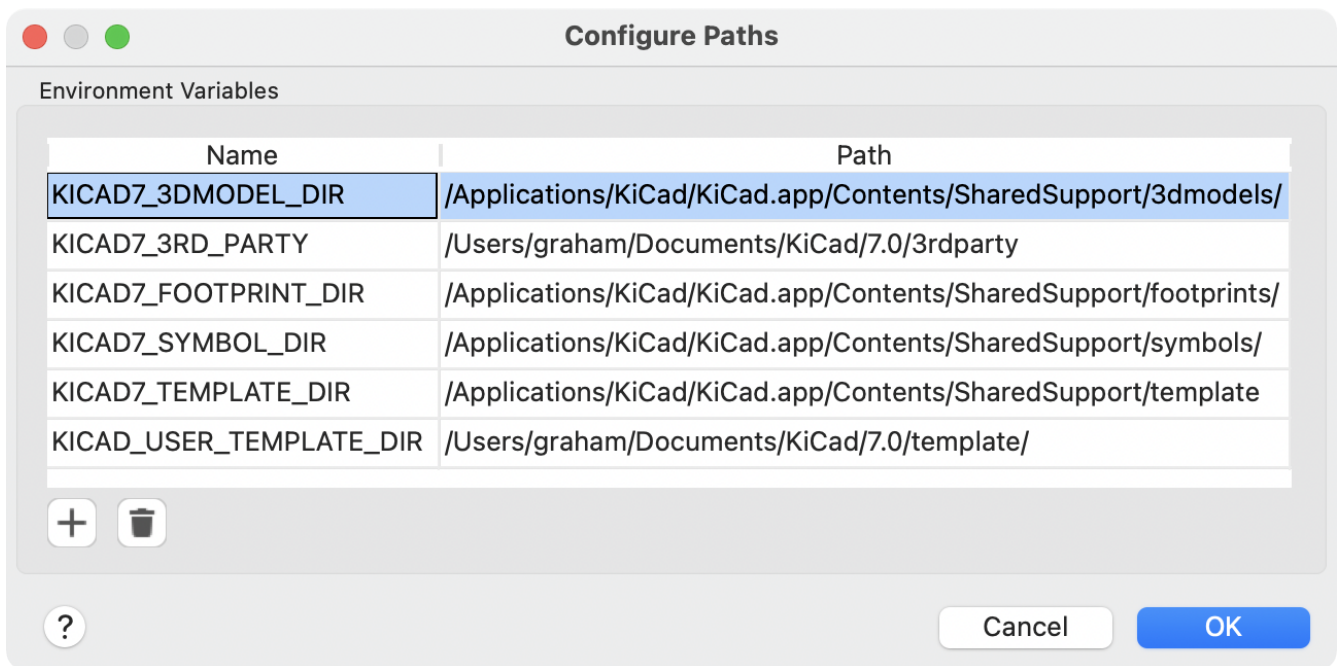
- Schematic symbol libraries
- Footprint libraries
- 3D model files used in footprint definitions

For instance, the path to the `connect.pretty` footprint library, when using the `KICAD7_FOOTPRINT_DIR` environment variable, would be defined as `${KICAD7_FOOTPRINT_DIR}/connect.pretty`.

The **Preferences** → **Configure Paths...** menu allows you to define paths for some built-in KiCad environment variables, and add your own environment variables to define personal paths, if needed.

KiCad environment variables

KICAD7_3DMODEL_DIR	Base path of KiCad's standard 3D footprint model library files. The KICAD6_3DMODEL_DIR variable automatically resolves to the same value if it is not separately defined.
KICAD7_3RD_PARTY	Location for plugins, libraries, and color themes installed by the Plugin and Content Manager .
KICAD7_FOOTPRINT_DIR	Base path of KiCad's standard footprint library files.
KICAD7_SYMBOL_DIR	Base path of KiCad's standard symbol library files.
KICAD7_TEMPLATE_DIR	Location of KiCad's standard project template library files.
KICAD_USER_TEMPLATE_DIR	Location of personal project templates.
SPICE_LIB_DIR	Location of personal simulation model libraries . This variable is not defined by default.



Paths set in the Configure Paths dialog are internal to KiCad and are not visible as environment variables outside of KiCad. They are stored in [KiCad's user configuration files](#).

Paths can also be set as environment variables outside of KiCad, which will override any settings in the user's configuration.

NOTE

You cannot override an environment variable that has been set outside of KiCad by using the Configure Paths dialog. Any variable that has been set externally will be shown as read-only in the dialog.

Note also that the environment variable `KIPRJMOD` is **always** internally defined by KiCad, and expands to the **current project absolute path**.

For instance, `${KIPRJMOD}/connect.pretty` is always the `connect.pretty` folder (the footprint library) inside **the current project folder**.

The `KIPRJMOD` variable cannot be changed in the Configure Paths dialog or overridden by an external environment variable.

Advanced environment variables

Some advanced environment variables can be set to customize KiCad's behavior. These variables are not shown in the environment variable configuration and cannot be used in path substitutions. They cannot be modified in the Configure Paths dialog, but they can be overridden by system environment variables.

Changing these variables will not result in KiCad moving any files from the default location to the new location, so if you change these variables you will need to copy any desired settings or files manually.

Additional environment variables:

KICAD_CONFIG_HOME	Base path of KiCad configuration files. Subdirectories will be created within this directory for each KiCad minor version.
KICAD_DOCUMENTS_HOME	Base path of KiCad user-modifiable documents, such as projects, templates, Python scripts, libraries, etc. Subdirectories will be created within this directory for each KiCad minor version. This directory is provided as a suggested user data location, but does not need to be used.

WARNING

If you modify the configuration of paths, please quit and restart KiCad to avoid any issues in path handling.

Libraries configuration

The **Preferences** → **Manage Symbol Libraries...** menu lets you manage the list of symbol libraries ([symbol library table](#)).

Likewise, use the **Preferences** → **Manage Footprint Libraries...** menu to manage the list of footprint libraries ([footprint library table](#)).

For each type of library (symbol and footprint), there are 2 library tables: global and project specific. The global library table is located in the [user configuration directory](#) and contains a list of libraries available to all projects. The project-specific library table is optional and contains a list of libraries specific to the project. It is located in the project directory.

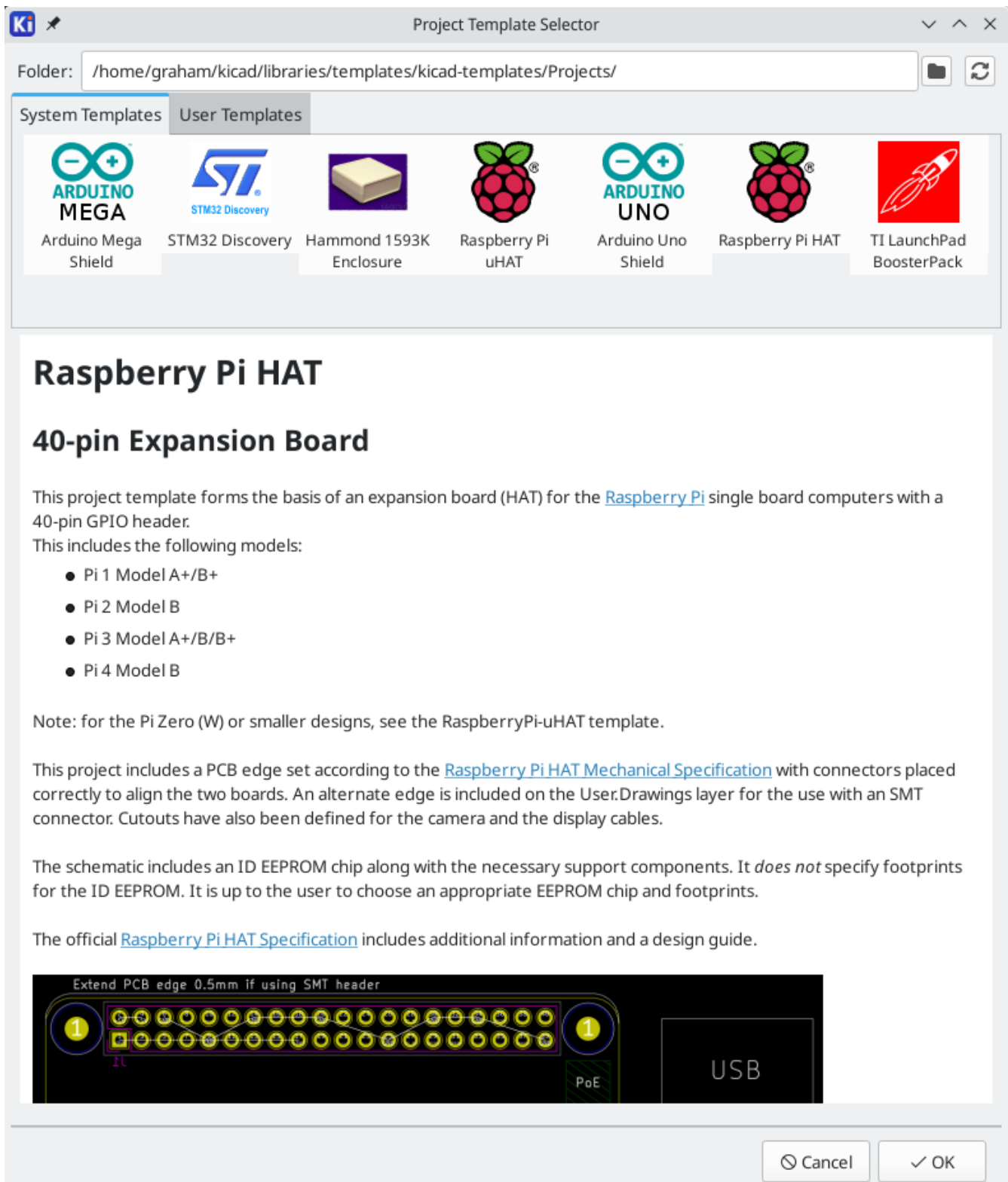
Project templates

Using a project template facilitates setting up a new project with predefined settings. Templates may contain pre-defined board outlines, connector positions, schematic elements, design rules, etc. Complete schematics and/or PCBs used as seed files for the new project may even be included.

Using templates

The **File** → **New Project from Template** menu will open the Project Template Selector dialog:

A single click on a template's icon will display the template information, and a further click on the OK button creates the new project. The template files will be copied to the new project location and renamed to reflect the new project's name.



Template locations

KiCad looks for template files in the following paths: KiCad looks for system templates in the path defined in the `KICAD7_TEMPLATE_DIR` path variable, and user templates in the path defined in `KICAD_USER_TEMPLATE_DIR`. However, you can browse for templates in an arbitrary directory using the **Folder** control at the top of the dialog.

Creating templates

The template name is the directory name where the template files are stored. The metadata directory is a subdirectory named `meta` containing files describing the template.

The metadata consists of one required file, and may contain optional files. All files must be created by the user using a text editor or previous KiCad project files, and placed into the required directory structure.

All files and directories in a template are copied to the new project path when a project is created using a template, except `meta`. Files and directories containing the template name will be renamed with the new project file name.

For example, creating a project called `newproject` from a template named `example`:

Files in template <code>example</code> directory	Files created in project <code>newproject</code> directory
<code>example.kicad_pro</code> <code>example.kicad_sch</code> <code>example.kicad_pcb</code> <code>example-first.kicad_sch</code> <code>second-example.kicad_sch</code> <code>third.kicad_sch</code> <code>third.kicad_pcb</code>	<code>newproject.kicad_pro</code> <code>newproject.kicad_sch</code> <code>newproject.kicad_pcb</code> <code>newproject-first.kicad_sch</code> <code>second-newproject.kicad_sch</code> <code>third.kicad_sch</code> <code>third.kicad_pcb</code>

A template does not need to contain a complete project. If a required project file is missing, KiCad will create it using its default create project behavior:

Files in template <code>example</code> directory	Files created in <code>newproject</code> directory
<code>example.kicad_sch</code> <code>first-example.kicad_sch</code> <code>first-example.kicad_pcb</code> <code>second-example.kicad_sch</code> <code>second-example.kicad_pcb</code>	<code>newproject.kicad_sch</code> <code>first-newproject.kicad_sch</code> <code>first-newproject.kicad_pcb</code> <code>second-newproject.kicad_sch</code> <code>second-newproject.kicad_pcb</code> <code>newproject.kicad_pro</code> (default) <code>newproject.kicad_pcb</code> (default)

As an exception to the template name renaming rule, if one project file (`.kicad_pro`) exists and its name doesn't match the template name, KiCad will do the renaming based on that project file name instead:

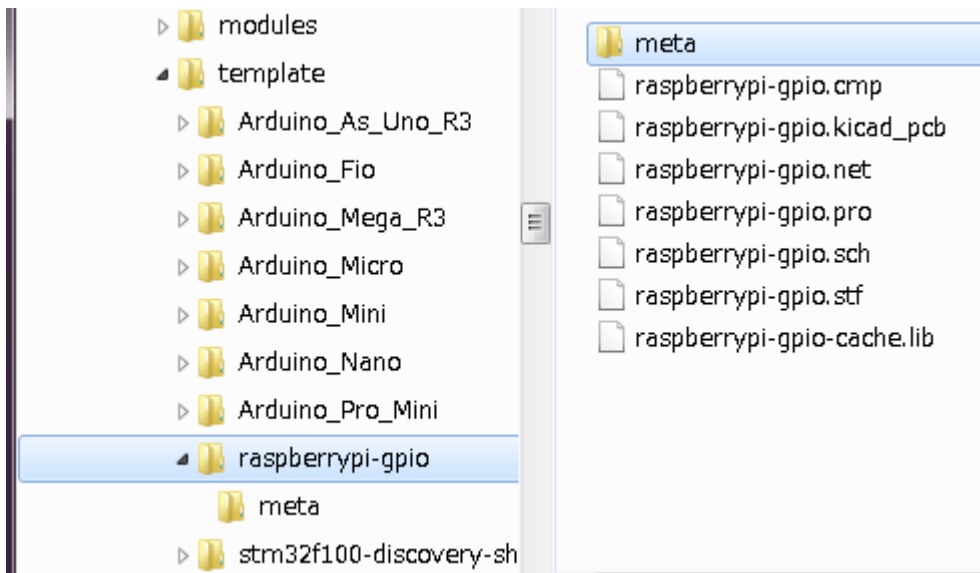
Files in template <code>example</code> directory	Files created in <code>newproject</code> directory
<code>example.kicad_sch</code> <code>example.kicad_pcb</code> <code>first-example.kicad_pro</code> <code>first-example.kicad_sch</code> <code>first-example.kicad_pcb</code> <code>second-example.kicad_sch</code> <code>second-example.kicad_pcb</code>	<code>example.kicad_sch</code> <code>example.kicad_pcb</code> <code>newproject.kicad_pro</code> <code>newproject.kicad_sch</code> <code>newproject.kicad_pcb</code> <code>second-example.kicad_sch</code> <code>second-example.kicad_pcb</code>

NOTE

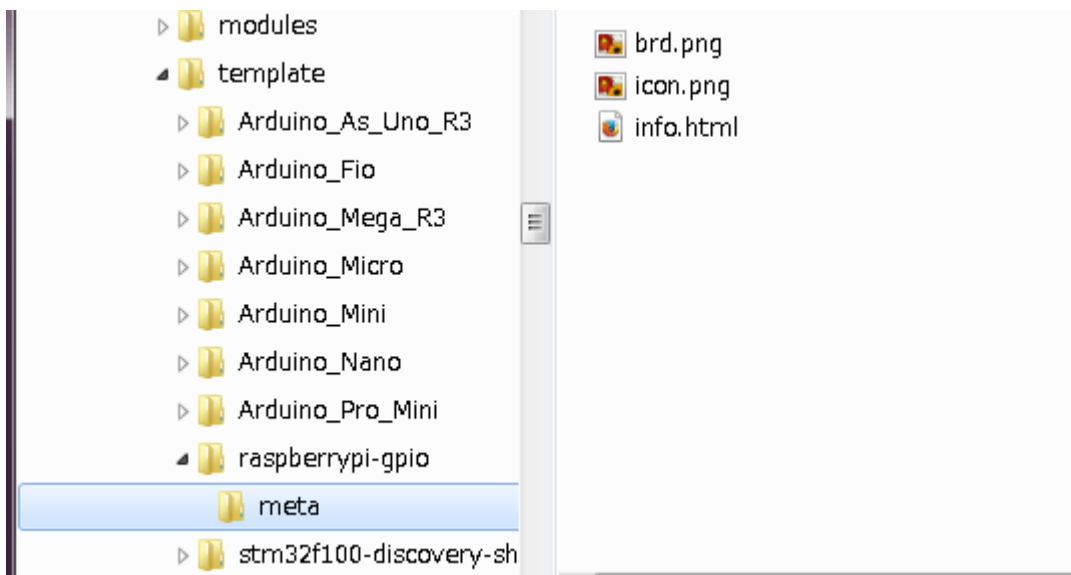
It is not recommended to create a template with multiple project files.

Template example

Here is an example showing project files for `raspberrypi-gpio` template:



And the metadata files:



Required file

meta/info.html	HTML-formatted information describing the template.
----------------	---

The `<title>` tag determines the actual name of the template that is exposed to the user for template selection. Note that the project template name will be cut off if it's too long.

Using HTML means that images can be easily in-lined without having to invent a new scheme. Only basic HTML tags can be used in this document.

Here is a sample `info.html` file:

```

<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.0 Transitional//EN">
<HTML>
<HEAD>
<META HTTP-EQUIV="CONTENT-TYPE" CONTENT="text/html;
charset=windows-1252">
<TITLE>Raspberry Pi - Expansion Board</TITLE>
</HEAD>
<BODY LANG="fr-FR" DIR="LTR">
<P>This project template is the basis of an expansion board for the
<A HREF="http://www.raspberrypi.org/" TARGET="blank">Raspberry Pi $25
ARM board.</A> <BR><BR>This base project includes a PCB edge defined
as the same size as the Raspberry-Pi PCB with the connectors placed
correctly to align the two boards. All IO present on the Raspberry-Pi
board is connected to the project through the 0.1" expansion
headers. <BR><BR>The board outline looks like the following:
</P>
<P><IMG SRC="brd.png" NAME="brd" ALIGN=BOTTOM WIDTH=680 HEIGHT=378
BORDER=0><BR><BR><BR><BR>
</P>
<P>(c)2012 Brian Sidebotham<BR>(c)2012 KiCad Developers</P>
</BODY>
</HTML>

```

Optional files

meta/icon.png	A 64 x 64 pixel PNG icon file which is used as a clickable icon in the template selection dialog.
---------------	---

Any other image files used by `meta/info.html`, such as the image of the board file in the dialog above, are placed in this folder as well.

Plugin and Content Manager

NOTE

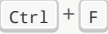
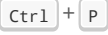
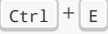
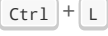
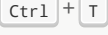
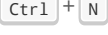
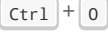
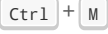
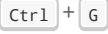
This section of the KiCad documentation has not yet been written. We appreciate your patience as our small team of volunteer documentation writers work to update and expand the documentation.

Actions reference

Below is a list of every available **action** in the KiCad Project Manager: a command that can be assigned to a hotkey.

KiCad Project Manager

The actions below are available in the KiCad Project Manager. Hotkeys can be assigned to any of these actions in the **Hotkeys** section of the preferences.

Action	Default Hotkey	Description
Close Project		Close the current project
Image Converter		Convert bitmap images to schematic or PCB components
Drawing Sheet Editor		Edit drawing sheet borders and title block
Footprint Editor		Edit PCB footprints
PCB Editor		Edit PCB
Schematic Editor		Edit schematic
Symbol Editor		Edit schematic symbols
New Project from Template...		Create new project from template
New Project...		Create new blank project
Open Demo Project...		Open a demo project
Open Project...		Open an existing project
Open Text Editor		Launch preferred text editor
Plugin and Content Manager		Run Plugin and Content Manager
Calculator Tools		Run component calculations, track width calculations, etc.
Gerber Viewer		Preview Gerber output files